

P1245R0

export module containing [
attribute];

Published Proposal, 2018-10-08

This version:

<http://wg21.link/p1245r0>

Issue Tracking:

[Inline In Spec](#)

Authors:

[Bruno Cardoso Lopes](#) (Apple)

[JF Bastien](#) (Apple)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/bcardosolopes/cpp-papers/blob/master/p1245r0.bs

Table of Contents

1 Introduction

2 Proposed Solution

2.1 Case study

References

Informative References

Issues Index

§ 1. Introduction

C++ supports *attributes* [dcl.attr.grammar] §9.11 that can be applied to source code in different ways:

Attribute syntax and semantics [dcl.attr.grammar]

¹ Attributes specify additional information for various source constructs such as types, variables, names, blocks, or translation units.

This is useful for fine-grained control of different capabilities and behaviors in a C++ codebase, but C++ currently lacks language-level constructs to express coarse-grained versions of the same mechanisms.

Compilers vend different kind of flags to control coarse-grained (translation unit level) capabilities such as ones that can affect C++ semantics, enforce restrictions, and the change the quality of code generation. Concrete examples:

Code generation

Global processor specific constraints in the lines of `-mmicromips`, `-mno-micromips` and `-msoft-float`. Global flags in [Clang] that change calling convention `-fdefault-calling-conv={cdecl,stdcall,vectorcall,regcall}`.

Efficiency and debuggability

The well known optimization level flags `-O0`, `-O1`, ... and debug `-g`

C++ semantics

Flags such as `-std=c++17`, or `-fno-rtti`, `-fno-exceptions` have the effect of changing or turning off parts of the standard.

Freestanding

The standard mentions *freestanding implementations* and *freestanding environment* [basic.start.main], but there's nothing obvious in user code that could enforce it as such, compiler flags like `-ffreestanding` are necessary.

§ 2. Proposed Solution

In the light of non-standard and implementation defined ways to specify such capabilities and constraints, we believe [Modules] are a great opportunity to bring some sanity to this madness and pave the road for incremental adoption of such flags in the form of *module attributes*.

Module attributes stand to remove out-of-band language modes, and make them official well-supported behavior. They're explicit in the code, removing sources of surprise. They standardize a pathway to express existing modes and also lower the number of incompatible options developers need to get from their compiler. They allow us to simplify our definition of freestanding.

According to [\[MergedModulesProposal\]](#) **[Basics]** §2.1:

... The `export` keyword indicates that a module unit is a module interface unit, which defines the interface for the module. For a module `foo`, there must be exactly one translation unit whose preamble contains `export module foo;`. This is the primary module interface unit for `foo`.

The rules for such *module attributes* are:

- They should be declared as part of *module interface unit*, more precisely at the end of the declaration that contains the `export` keyword, before `;`.

```
export module foo [[shiny_new_attribute]];
import a;
export import b;
// ... more imports ...
```

- Existing attributes with fine-grained scopes (types, variables, names, etc) can be modified on a case-by-case basis by the Committee to also provide module-level behavior, as long as the behavior is clearly specified in the Standard. Example: a function level attribute used at the module level could turn on a capability or restriction for all functions in the module.
- Attributes at the module-level can be overridden by finer-grained attributes.
- Conflicting module-level and other fine-grained scope attributes should be diagnosed by the compiler.
- Some compiler settings could *require* that some module-level attributes be used. For example, targetting an embedded platform which has no support for floating-point could hard-error on any module which doesn't contain the `[[no_float]]` attribute.

ISSUE 1 It's yet unclear how [\[Contracts\]](#) are going to interoperate with Modules, but this proposal certainly touches some aspect of it that should be considered in the future.

§ 2.1. Case study

Take the `[[no_float]]` *function attribute* described in [\[P1246R0\]](#). As the mentioned rationale in **[Introduction]**:

- Different pieces of a project might want different floating-point code generation; relying on extra compiler flags for specific translation units in a project is brittle and also has maintenance and portability costs. For instance, in an application one might want to opt out from floating-point only for specific pieces (e.g. cryptography code and kernel level `printf`-like utilities still want to use floating-point)

Using `[[no_float]]` as a *module attribute* is a great fit for the requirements above. Based on the example above:

```
export module Crypto [[no_float]];
import BigNum;
...
export class MyBlockCipher { ... }
...
```

This hypothetical cryptography code lives on its own module and is marked with `[[no_float]]`, meaning every function inside the `Crypto` module shall not use floating-point (here, maybe vectorization and custom-hardware crypto is undesirable and only integer instructions are desired). Using this annotation yield some benefits:

Readability expectations

It's clear from the module interface that one isn't expected to use floating-point. The code is also clean, since there's no need to annotate every function with such attribute.

Policy enforcement

Attempts to use floating-point code will hard-error on the user.

Additionally, `printf`-like utilities, in their own module, can still use floating-point and be imported by the same translation unit that imports `Crypto`.

§ References

§ Informative References

[BASIC.START.MAIN]

C++ working draft: 6.8.3.1 main function. URL: <http://eel.is/c++draft/basic.start#main-1>

[Clang]

Clang: a C language family frontend for LLVM. URL: <http://clang.llvm.org>

[Contracts]

Contracts attributes. URL: <http://eel.is/c++draft/dcl.attr.contract>

[MergedModulesProposal]

Merging Modules. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1103r0.pdf>

[Modules]

Modules for Standard C++. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1087r0.pdf>

[P1246R0]

The no_float function attribute. URL: <http://wg21.link/p1246r0>

§ Issues Index

ISSUE 1 It's yet unclear how [\[Contracts\]](#) are going to interoperate with Modules, but this proposal certainly touches some aspect of it that should be considered in the future. ↩